Introduction

LLM routing has become increasingly important for AI-assisted software platforms. When using coding agents, I noticed that some prompts clearly work better on certain models, but stronger models are also much more expensive. The AI systems of today have many different models each with differing performance and costs, usually with a general positive correlation. However, because this correlation is usually non-linear, and companies must also face real life factors such as budget constraints and consumer expectations, I will be modeling such a problem to solve.

In this project, I study the routing problem from the perspective of an IDE company focused on coding tasks. The company wishes to maximize response quality while maintaining reasonable costs. I formulate the problem as a two-stage stochastic optimization model. In the first stage, the company selects a limited pool of models for deployment. In the second stage, each incoming prompt is routed to one of the selected models.

The project investigates the following two research questions. I focused on these because they seemed to be baseline for practical relevancy for a company deciding how many models to maintain and how much inference cost is actually worth paying:

1. How large should the model pool be before performance gains begin to diminish?
2. How sensitive is routing performance to the inference budget?

To answer these questions, I build a constrained routing optimization model using Pyomo and evaluate it on the RouterBench dataset.

Optimization Model

I formulate the routing problem as a two-stage stochastic optimization problem. In the first stage, the company selects a subset of models to maintain in the deployment pool. In the second stage, each prompt is assigned to **one** selected model. Prompt requests are treated as samples from an unknown distribution, so I use a weighted sample-average approximation to estimate expected routing performance.

Sets and Parameters:

P := {p} Set of prompts
M := {m} Set of candidate models
q_pm Performance score when prompt p is answered by model m
c_pm Cost when prompt p is answered by model m
w_p Weight associated with prompt p
K Maximum allowable model pool size
B Average budget
LCB_min Minimum required coding benchmark score

Decision Variables:
x_pm $\in$ **{0,1}** Binary variable indicating whether prompt p is routed to model m
y_m $\in$ **{0,1}** Binary variable indicating whether model m is selected in the pool

Objective Function:
The objective of the model is to maximize the weighted average routing performance across all prompts:

$$\max \sum_{p \in P} w\_p \sum_{m \in M} (q\_pm * x\_pm)$$

The objective attempts to maximize overall response quality while the constraints control deployment size, budget, and coding reliability. Since prompts are treated as samples from an underlying prompt distribution, the weighted objective acts as a sample-average approximation of expected routing performance.

Constraints:

Prompt Assignment Constraint: $\sum_{m \in M} x\_pm = 1; \ \forall p \in P$

A prompt can only be routed to a model if that model was selected in the deployment pool.

Pool Size Constraint: $\sum_{m \in M} y\_m <= K$

The company may deploy at most K models.

Budget Constraint: $\sum_{p \in P} w\_p \sum_{m \in M} (c\_pm * x\_pm) <= B$

The weighted average inference cost must be below the deployment budget B.

Quality Constraint: $\frac{1}{|P_{LCB}|} \sum_{p \in P_{LCB}} \sum_{m \in M} (q\_pm*x\_pm) >= L\_min$

The average routing score on coding prompts must remain above a minimum threshold. I used L_min=0.80 in the experiments to maintain acceptable coding quality.

Stochastic Interpretation and Sample-Average Approximation:

The routing problem is stochastic because future prompt requests are uncertain. In practice, the company does not know the exact distribution of future prompts submitted by users. Instead, the RouterBench dataset is treated as a finite sample from an underlying population distribution of prompts.

To approximate expected routing performance, I use a weighted sample-average approximation. Each prompt p is assigned a weight w_p and the objective maximizes the weighted average routing score across the sampled prompts.

The population-level optimization problem can be viewed as maximizing expected routing performance: **max E[Q(p, m)]** where Q(p,m) is the routing performance when prompt p is assigned to model m

Since the true population distribution is unknown, I approximate this expectation using

the empirical dataset: **max** $\sum_{p \in P}$ w_p $\sum_{m \in M}$ (q_pm*x_pm)

This converts the stochastic routing problem into a solvable finite optimization problem.

Pyomo

Implementation:

The optimization model was implemented in Pyomo using binary decision variables for both model selection and prompt routing. The HiGHS solver was used to solve the resulting mixed-integer optimization problem.

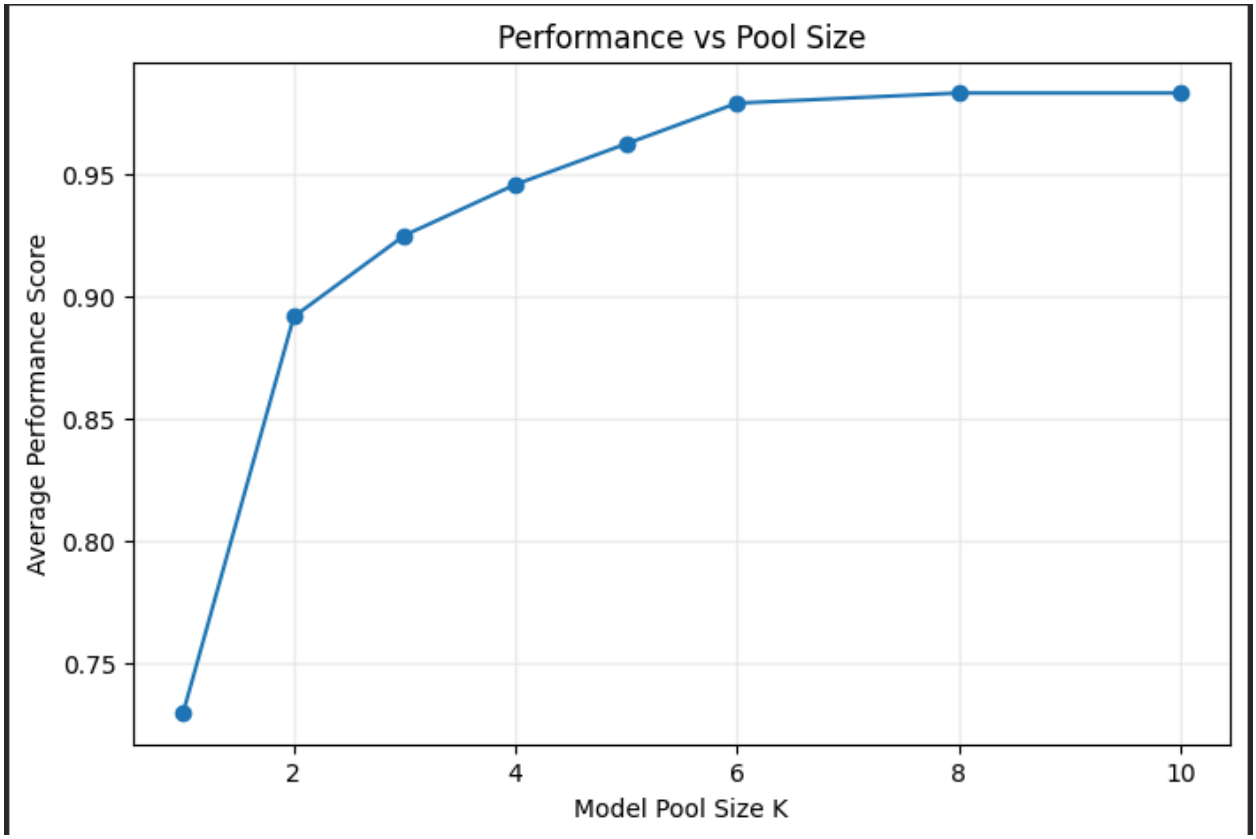
The implementation includes: model pool selection variables, prompt routing variables, budget constraints, quality constraints, and parameter sweeps for pool size and budget experiments.

Results:

Experiment 1: Pool Size Sweep

In the first experiment, I fixed the deployment budget at B = 0.005 and the minimum coding benchmark threshold at L_min = 0.80. I then solved the optimization model for different maximum model pool sizes: K ∈ {1, 2, 3, 4, 5, 6, 8,10}. The goal of this experiment was to determine how many deployed models are needed before routing performance begins to show diminishing returns.

K	budget	min_lcb_score	avg_cost	avg_score	avg_lcb_score
1	0.005	0.8	0.001756	0.729167	0.800000
2	0.005	0.8	0.004886	0.891667	0.933333
3	0.005	0.8	0.005000	0.925000	0.983333
4	0.005	0.8	0.004996	0.945833	0.983333
5	0.005	0.8	0.004998	0.962500	1.000000
6	0.005	0.8	0.004690	0.979167	1.000000
8	0.005	0.8	0.004606	0.983333	1.000000
10	0.005	0.8	0.005000	0.983333	1.000000



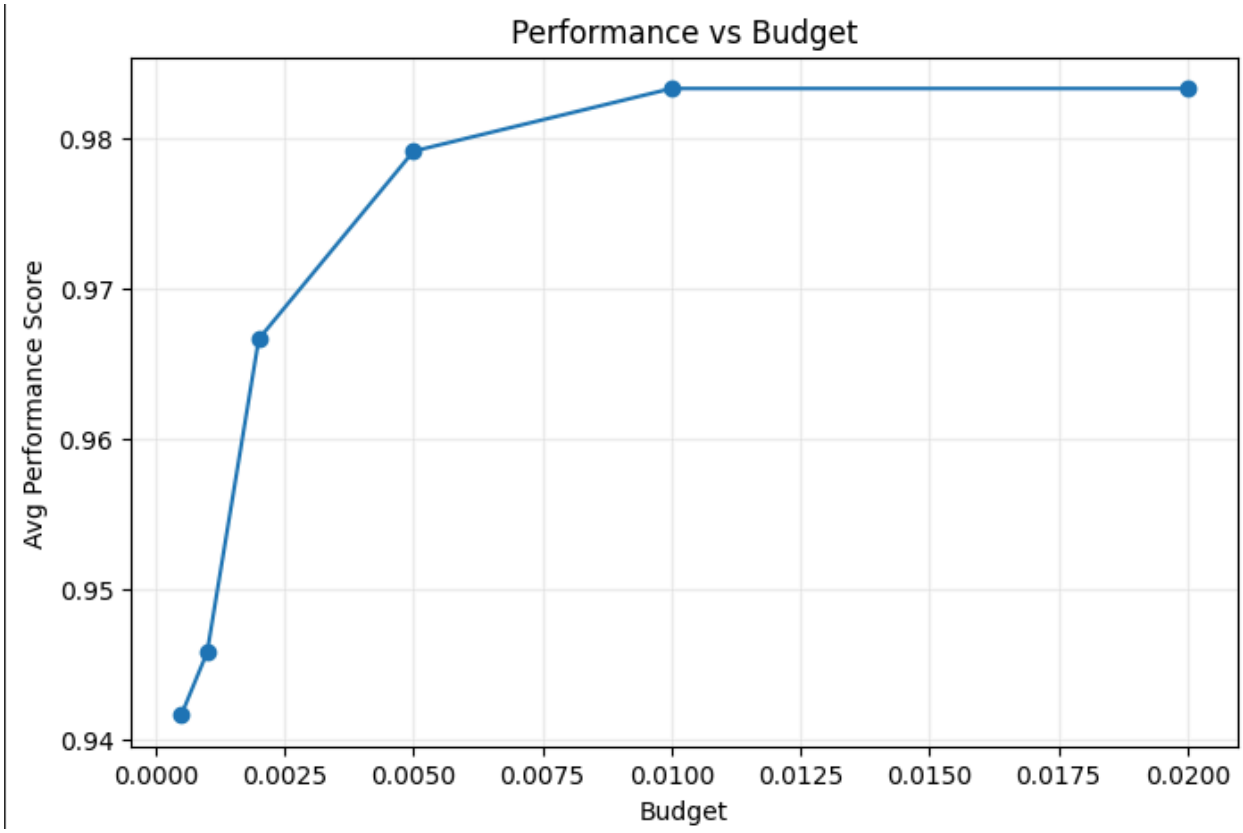
The results show that routing performance improved rapidly as the model pool increased from one model to several models. The jump from K = 1 to K = 2 was especially large, with the average routing score increasing from approximately 0.73 to 0.89. However, after around K = 6, the improvements became much smaller. The average score increased from approximately 0.979 at K = 6, to only 0.983 at K = 8 with no further improvement at K = 10. This suggests strong diminishing returns beyond 6 to 8 deployed models. Overall, the experiment suggests that a moderate-sized curated model pool can achieve near-optimal routing performance without requiring deployment of all candidate models.

Experiment 2: Budget Sweep

After identifying diminishing returns around K = 6, I fixed the deployment pool size at six models and studied how routing quality changes as the deployment budget varies. I also fixed the minimum coding benchmark threshold at L_min = 0.80. I then solved the optimization model for the following budget levels:

B ∈ {0.0005, 0.001, 0.002, 0.005, 0.01, 0.02}. The purpose of this experiment was to study how sensitive routing performance is to cost constraints.

	K	budget	min_lcb_score	avg_cost	avg_score	avg_lcb_score
0	6	0.0005	0.8	0.000477	0.941667	0.966667
1	6	0.0010	0.8	0.001000	0.945833	0.950000
2	6	0.0020	0.8	0.001899	0.966667	0.983333
3	6	0.0050	0.8	0.004690	0.979167	1.000000
4	6	0.0100	0.8	0.009978	0.983333	1.000000
5	6	0.0200	0.8	0.019699	0.983333	1.000000



The results show that routing performance generally improved as the budget increased. Under the tightest budget of 0.0005, the average routing score was approximately

0.942. As the budget increased to 0.01, the average score improved to approximately 0.983.

Interestingly, the performance gains became very small beyond a budget of around 0.01. Increasing the budget from 0.01 to 0.02 produced essentially no additional improvement. I originally expected the score to continue increasing more noticeably at higher budgets, but the results plateaued earlier than I anticipated.

The selected model pools also changed substantially across budget levels. Under tighter budgets, the optimizer relied more heavily on lightweight and open-source models such as MiniCPM4.1-8B and NVIDIA-Nemotron-Nano-9B-v2. GPT-5 did not appear in the selected pool under the tightest budgets and only entered the optimal pool once the budget increased to approximately 0.005.

```
Budget: 0.0005
Intern-S1-mini; Llama-3.1-8B-Instruct; MiniCPM4.1-8B; NVIDIA-Nemotron-Nano-9B-v2; deepseek-v3-0324; qwen3-235b-a22b-2507

Budget: 0.001
Intern-S1-mini; Llama-3.1-8B-Instruct; NVIDIA-Nemotron-Nano-9B-v2; deepseek-v3-0324; kimi-k2-0905; qwen3-235b-a22b-2507

Budget: 0.002
GLM-Z1-9B-0414; Intern-S1-mini; Llama-3.1-8B-Instruct; deepseek-v3-0324; kimi-k2-0905; qwen3-235b-a22b-thinking-2507

Budget: 0.005
GLM-Z1-9B-0414; Llama-3.1-8B-UltraMedical; glm-4.6; gpt-5; kimi-k2-0905; qwen3-235b-a22b-thinking-2507

Budget: 0.01
GLM-Z1-9B-0414; Llama-3.1-8B-UltraMedical; gemini-2.5-flash; glm-4.6; gpt-5; kimi-k2-0905

Budget: 0.02
GLM-Z1-9B-0414; Llama-3.1-8B-UltraMedical; gemini-2.5-flash; glm-4.6; gpt-5; kimi-k2-0905
```

These results suggest that budget aware routing policies can achieve strong performance while controlling deployment costs, and that excessive spending may provide limited marginal benefit once sufficiently strong models become available at certain costs.

Discussion and Recommendations:

The optimization model successfully answers both research questions posed in the introduction.

First, the pool-size experiment demonstrated strong diminishing returns beyond approximately six deployed models. This suggests that companies may not need to maintain extremely large model pools in order to achieve high routing performance. A smaller curated pool can already provide near optimal results while reducing engineering overhead and maintenance burden.

Second, the budget experiment showed that routing quality improves as inference spending increases, but eventually saturates once sufficiently strong models become available. Beyond approximately budget 0.01, additional spending produced almost no measurable improvement in routing score.

Based on these findings, I would recommend that the company deploy a moderate sized model pool combined with a budget aware routing strategy. Simpler prompts can be handled by lightweight or open-source models, while more difficult prompts can be routed to stronger proprietary models when necessary. This allows the company to balance cost efficiency with response quality.

Personal Takeaway:

I thought this project was interesting because it connected optimization to something very current and practical. Before working on this, I mostly thought about LLMs from the perspective of model performance, but this project made me think more about the deployment side and the tradeoffs companies actually have to consider. It was interesting seeing how the optimizer would sometimes prefer combinations of cheaper models instead of always choosing the strongest models available. I enjoyed working on the project because it felt more realistic than a standard optimization assignment and connected well to real AI systems being used today.

Full Pyomo implementation + experimentation link:

https://drive.google.com/file/d/1z35MjVxMOCogF2AgwfNh5NPlb29P_K-A/view?usp=sharing